Exp no - 6
Date - 08/08/2026
Name - Andryan Durai Jebin

ERROR CORRECTION AT DATA LINK LAYER USING HAMMING CODE

AIM:

To write a program to implement error detection and correction using the Hamming Code concept and verify the error correction capability by transmitting a data stream from sender to receiver.

ALGORITHM:

SENDER:

1. Read a text message from the user.
2. Convert each character of the message into its 8-bit binary representation.
3. Divide the binary data into suitable blocks for Hamming encoding.
4. Calculate the number of redundant bits required for each block.
5. Insert the redundant bits at positions 1, 2, 4, 8, and so on.
6. Calculate the parity bits using even parity.
7. Generate the Hamming-encoded binary data.
8. Save the encoded data into a file named channel.

RECEIVER:

1. Read the encoded binary data from the channel file.
2. Calculate the parity bits again to check for transmission errors.
3. Determine the error position using the syndrome value.
4. If an error is detected, correct the bit at the identified position.
5. Remove the redundant parity bits from the corrected data.
6. Convert the recovered binary data into ASCII characters.
7. Display the original message.

PROGRAM:

SENDER PROGRAM (sender.py)

```
import math

message = input("Enter the message: ")

binary = ''.join(format(ord(ch), '08b') for ch in message)
```

```
encoded = []

for start in range(0, len(binary), 4):
 data = binary[start:start + 4].ljust(4, '0')
 n = 7
 code = [0] * (n + 1)

positions = [3, 5, 6, 7]

for i, pos in enumerate(positions):
 code[pos] = int(data[i])

for p in [1, 2, 4]:
 parity = 0
 for i in range(1, n + 1):
 if i & p and i != p:
 parity ^= code[i]
 code[p] = parity

encoded.append(''.join(map(str, code[1:])))

with open("channel", "w") as f:
 f.write('\n'.join(encoded))

print("Encoded data saved to channel file.")
print("Encoded data:")
print('\n'.join(encoded))
```

RECEIVER PROGRAM (receiver.py)

```
with open("channel", "r") as f:
 blocks = f.read().splitlines()

decoded = ""

for block in blocks:
 if len(block) != 7 or any(bit not in '01' for bit in block):
 print("Invalid data block")
 continue

code = [0] + [int(bit) for bit in block]

syndrome = 0
```

```
for p in [1, 2, 4]:
 parity = 0
 for i in range(1, 8):
 if i & p:
 parity ^= code[i]
 if parity:
 syndrome += p

if syndrome:
 print("Error detected at position:", syndrome)
 code[syndrome] ^= 1
 print("Error corrected.")

data = ''.join(str(code[i]) for i in [3, 5, 6, 7])
 decoded += data

decoded = decoded[:(len(decoded) // 8) * 8]

message = ''.join(
 chr(int(decoded[i:i + 8], 2))
 for i in range(0, len(decoded), 8)
)

print("Received message:", message)
```

PROCEDURE:

1. Save the sender program as sender.py and the receiver program as receiver.py in the same folder.
2. Run the sender program using the command python sender.py.
3. Enter a text message when prompted.
4. The encoded binary data will be stored in the channel file.
5. Run the receiver program using the command python receiver.py.
6. Observe the received message and verify that it matches the original message.

OUTPUT:

Sender:

Enter the message: HELLO

Encoded data saved to channel file.

Receiver:

Received message: HELLO

RESULT:

Thus, the Hamming Code technique was implemented to perform error detection and correction at the data link layer. The encoded data was stored in a channel file, decoded at the receiver, and the original message was successfully recovered.